

Deep Learning (CAI-461)

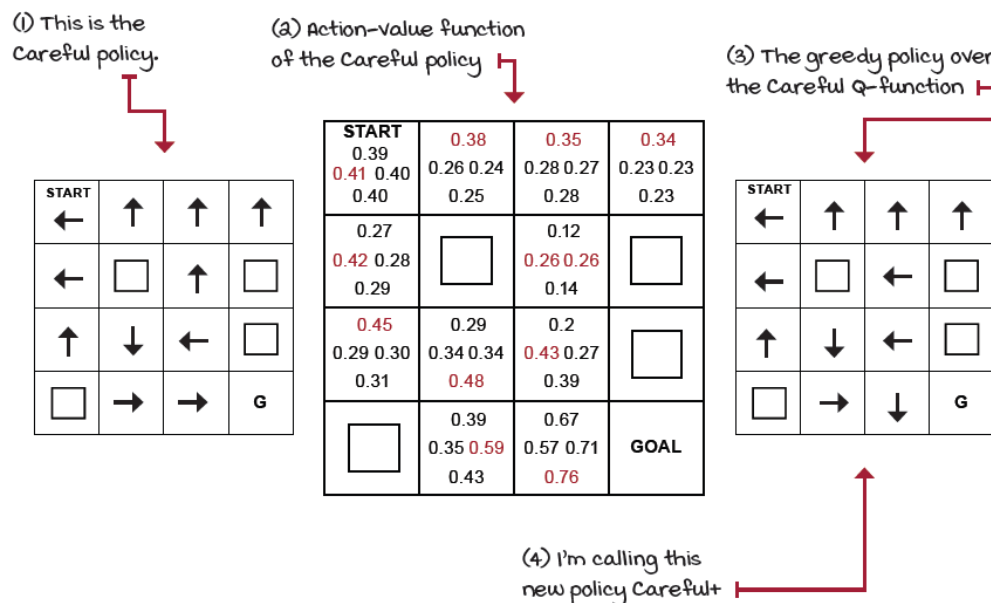
Objectives:

- **Policy Improvement**
- ✓ **Policy iteration method**
- ✓ **Value Iteration method**

Policy improvement

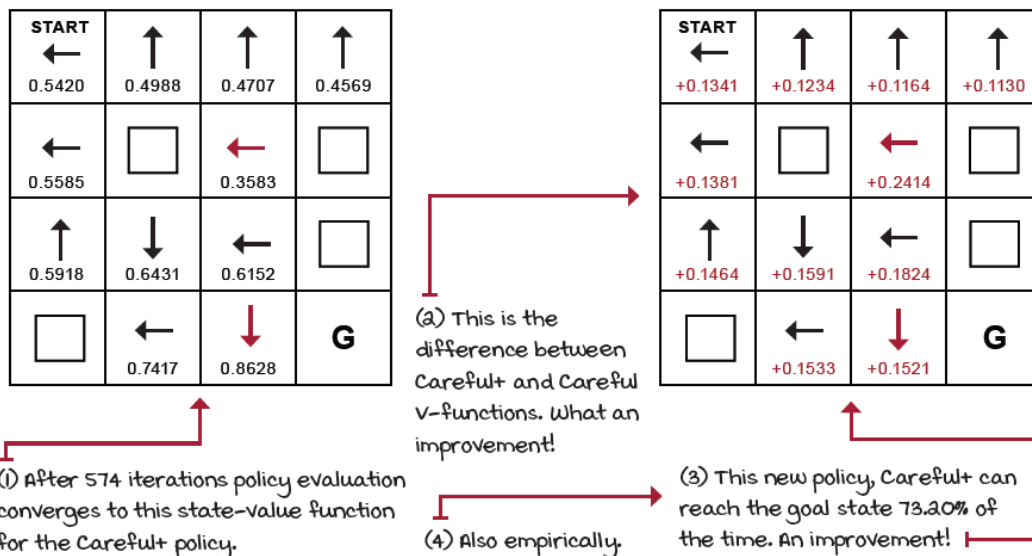
The motivation is clear now. You have a way of evaluating any policy. This already gives you some freedom: you can evaluate many policies and rank them by the state-value function of the START state. After all, that number tells you the expected cumulative reward.

The key to unlocking this problem is the action-value function, the Q-function. Using the V-function and the MDP, you get an estimate of the Q-function. The Q-function will give you a glimpse of the values of all actions for all states, and these values, in turn, can hint at how to improve policies. Take a look at the Q-function of the Careful policy and ways we can improve this policy:



Notice how if we act greedily with respect to the Q-function of the policy, we obtain a new policy: Careful+.

State-value function of the Careful policy



The new policy is better than the original policy. This is great! We used the state-value function of the original policy and the MDP to calculate its action-value function. Then, acting greedily with respect to the action-value function gave us an improved policy. This is what **the policy-improvement** algorithm does: it calculates an action-value function using the state-value function and the MDP, and it returns a *greedy* policy with respect to the action-value function of the original policy.

(1) To improve a policy, we use a state-value function and an MDP to get a one-step look-ahead and determine which of the actions lead to the highest value. This is the policy-improvement equation.

(2) We obtain a new policy π' by taking the highest-valued action.

(3) How do we get the highest-valued action?

$$\pi'(s) = \operatorname{argmax}_a \sum_{s', r} p(s', r | s, a) [r + \gamma v_{\pi}(s')]$$

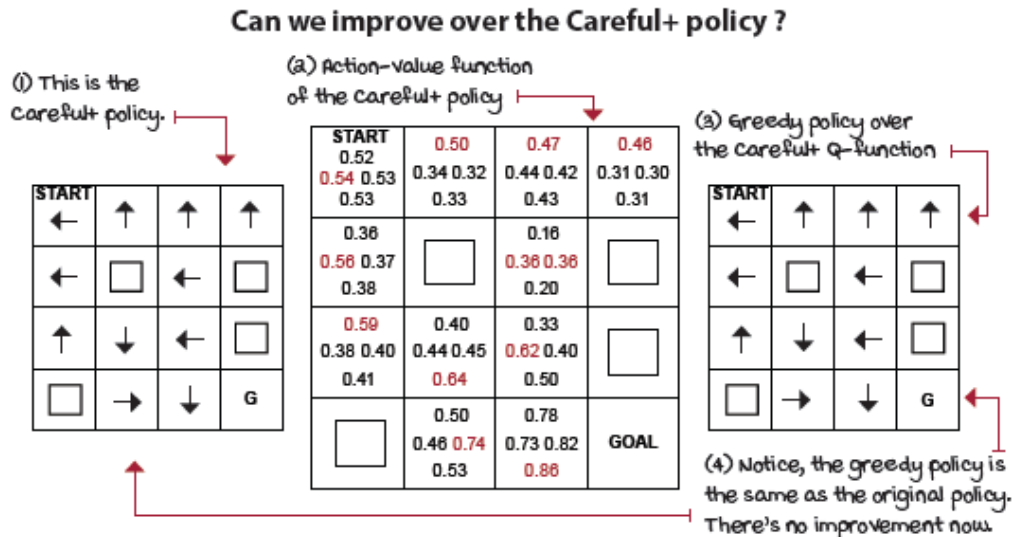
(4) By calculating, for each action, the weighted sum of all rewards and values of all possible next states.

(5) Notice that this uses the action with the highest-valued Q-function.

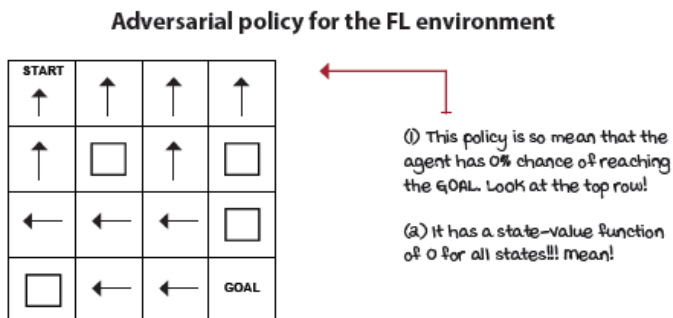
Can we improve a policy let's see below.

I ran policy evaluation on the Careful+ policy, and then policy improvement. The Q-functions of Careful and Careful+ are different, but the greedy policies over the Q-functions are identical. In other words, there's no improvement this time.

No improvement occurs because the Careful+ policy is an optimal policy of the FL environment (with gamma 0.99). We only needed one improvement over the Careful policy because this policy was good to begin with.

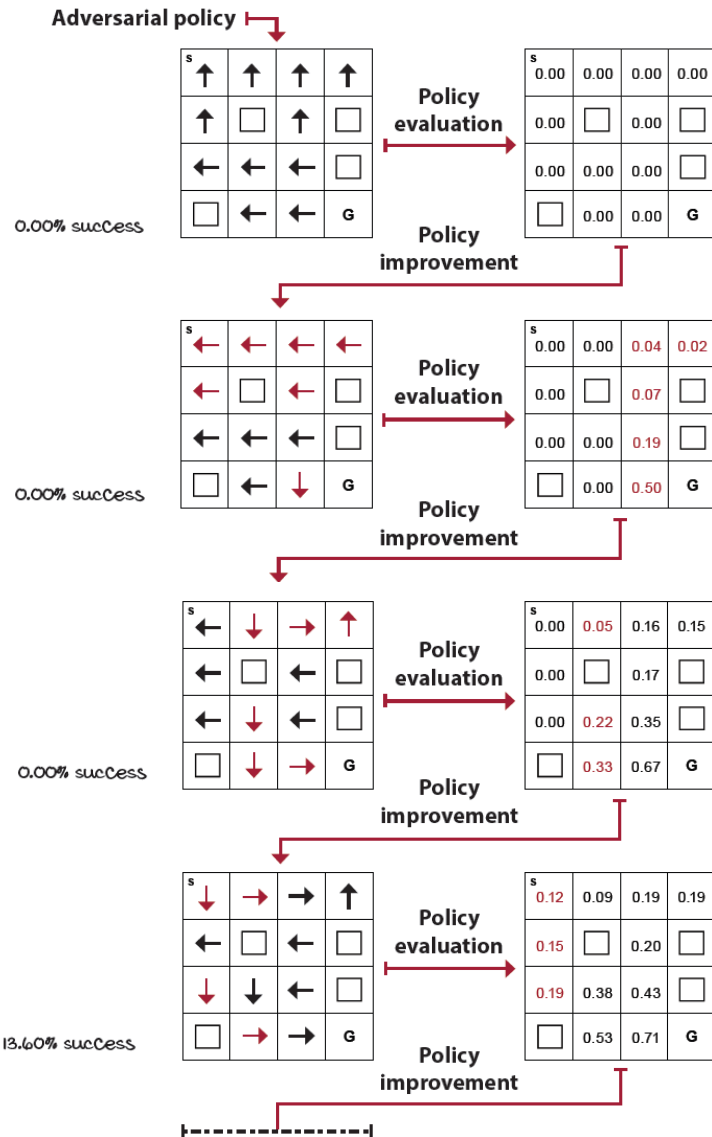


Now, even if we start with an adversarial policy designed to perform poorly, alternating over policy evaluation and improvement would still end up with an optimal policy. Let's make up an adversarial policy for FL environment and see what happens.



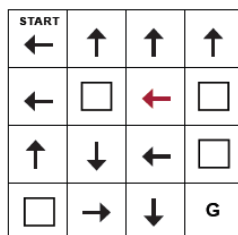
Policy iteration:

The plan with this adversarial policy is to alternate between policy evaluation and policy improvement until the policy coming out of the policy-improvement phase no longer yields a different policy. The fact is that, if instead of starting with an adversarial policy, we start with a randomly generated policy, this is what an algorithm called *policy iteration* does. Great! But, let's first try it starting with the adversarial policy and see what happens.



The FL environment has multiple optimal policies

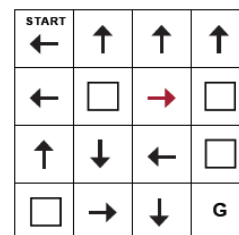
(1) Optimal action-value function



(2) A policy going left in state b_6 is optimal!

START	0.52	0.50	0.47	0.46
0.54	0.53	0.34	0.32	0.44
0.53		0.33	0.43	0.31
0.36			0.16	
0.56	0.37		0.36	0.36
0.38			0.20	
0.59	0.40	0.33		
0.38	0.40	0.44	0.45	0.62
0.41		0.64	0.50	
	0.50	0.78		
	0.46	0.74	0.73	0.82
	0.53	0.86		GOAL

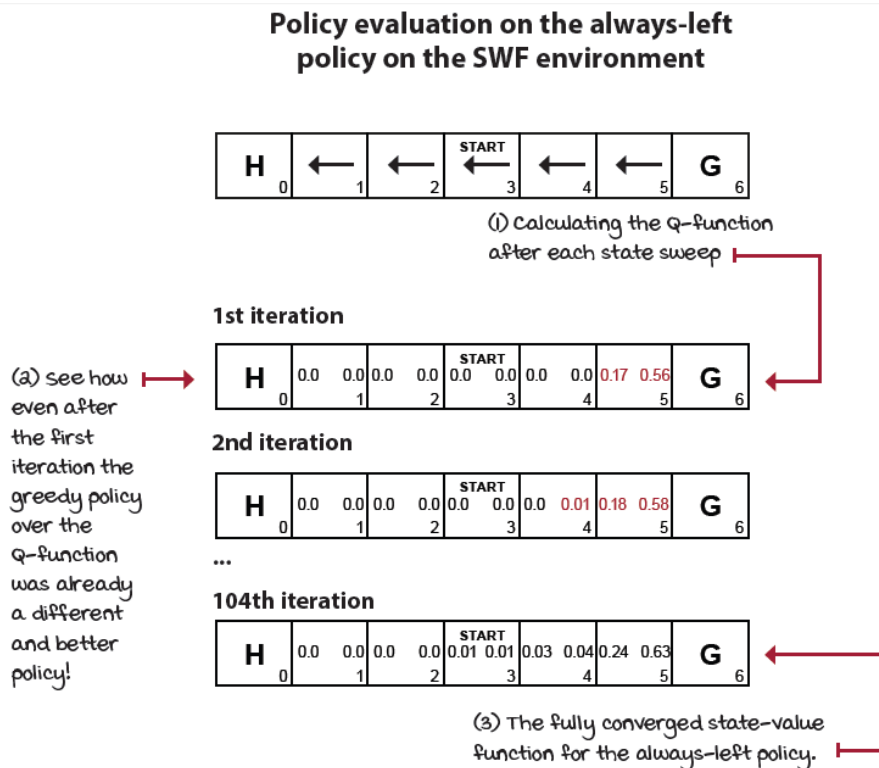
(3) But, look at state b_6 .



(4) Here's a policy that goes right in state b_6 , and it's as good, and also optimal!

Value iteration:

You probably notice the way policy evaluation works: values propagate consistently on each iteration, but slowly. Take a look.

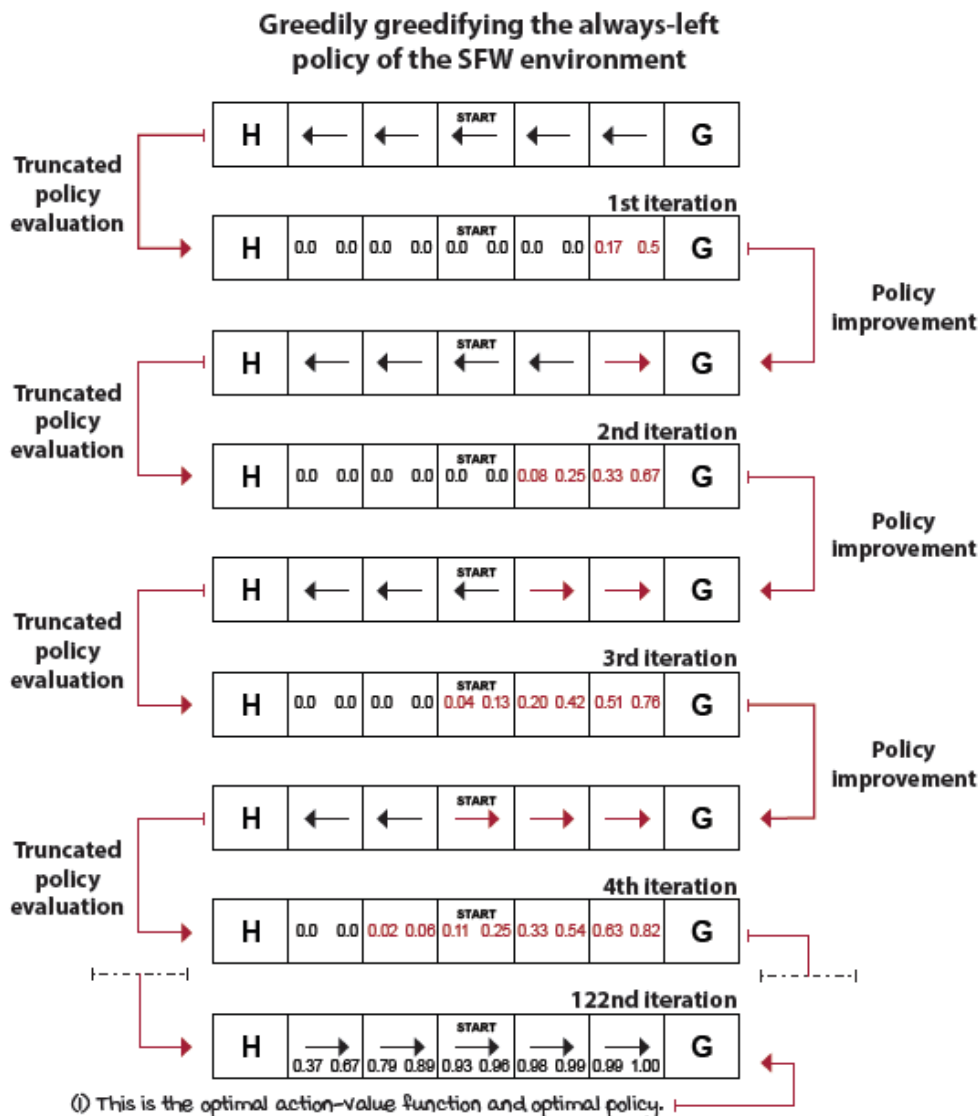


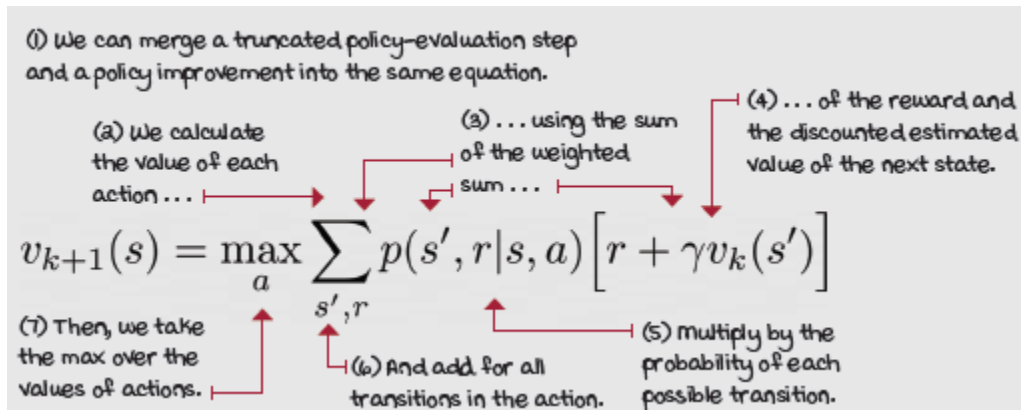
The image shows a single state-space sweep of policy evaluation followed by an estimation of the Q-function. We do this by using the truncated estimate of the V-function and the MDP, on each iteration. By doing so, we can more easily see that even after the first iteration, a greedy policy over the early Q-function estimates would be an improvement. Look at the Q-values for state 5 in the first iteration; changing the action to point towards the GOAL state is obviously already better.

In other words, even if we truncated policy evaluation after a single iteration, we could still improve upon the initial policy by taking the greedy policy of the Q-function estimation after a single state-space sweep of policy evaluation. This algorithm is another fundamental algorithm in RL: it's called *value iteration* (VI).

VI can be thought of “greedily greedifying policies,” because we calculate the greedy policy as soon as we can, greedily. VI doesn’t wait until we have an accurate estimate of the policy before it improves it, but instead, VI truncates the policy-evaluation phase after a single state sweep. Take a look at what I mean by “greedily greedifying policies.”

If we start with a randomly generated policy, instead of this adversarial policy always-left for the SWF environment, VI would still converge to the optimal state-value function. VI is a straightforward algorithm that can be expressed in a single equation.



Value iteration equation

If we start with a randomly generated policy, instead of this adversarial policy always-left for the SWF environment, VI would still converge to the optimal state-value function. VI is a straightforward algorithm that can be expressed in a single equation.